

Contexto Completo do Projeto

Sistema dinamico para probabilidades, regras de associacao e programacao linear.

1. Contexto do trabalho

O projeto foi desenvolvido a partir do enunciado do professor Jose Carlos Ferreira da Rocha. A proposta era usar uma base com variaveis categoricas para extrair conhecimento probabilistico, representar esse conhecimento como restricoes de um programa linear e permitir que o usuario fizesse perguntas condicionais do tipo $P(A | B)$.

2. Objetivo implementado

Foi construida uma aplicacao web em Python/Flask com frontend dinamico. O usuario escolhe afirmacoes e uma pergunta na interface. O backend calcula probabilidades, monta o modelo linear, resolve com solver e retorna metricas, intervalo linear, tempo de processamento e conclusao automatica.

3. Dataset usado

Item	Descricao
Arquivo	data/Crop_recommendation.csv
Origem conceitual	Dataset de recomendacao de culturas agricolas usado como base de solo/cultura.
Colunas	N, P, K, temperature, humidity, ph, rainfall, label
Tratamento	Colunas numericas sao categorizadas automaticamente; label permanece como classe/cultura.
Dinamismo	O sistema detecta as colunas do CSV e monta a interface com os dominios reais da base.

4. Arquitetura do sistema

Camada	Responsabilidade	Arquivos
Frontend	Interface para escolher A, B e visualizar resultados.	index.html, styles.css, script.js
API Python	Calcula probabilidades, valida consulta e chama solver.	app.py
Solver	Resolve o programa linear via SciPy linprog/HiGHS.	requirements.txt, app.py
Relatorios	Gera PDFs e JSONs com explicacoes e saidas calculadas.	reports/
Deploy	Configura Render com Python 3.11.9 e Unicorn.	render.yaml, Procfile, .python-version
Compatibilidade	Alias WSGI para start command antigo do Render.	your_application/wsgi.py

5. Fluxo de uso

1. O usuario abre a interface.
2. A interface consulta `/api/metadata` para buscar atributos e valores do dataset.
3. O usuario escolhe o evento A e as afirmacoes B.
4. A interface envia a consulta para `/api/query`.
5. O backend calcula $P(A)$, $P(B)$, $P(A \text{ e } B)$, suporte, confianca e lift.
6. O backend monta restricoes lineares e resolve o intervalo de $P(A | B)$.
7. A interface mostra resultado, programa linear, tempo e conclusao.

6. Matematica do projeto

Conceito	Formula	Uso
Marginal	$P(A) = \text{count}(A) / N$	Probabilidade de um valor isolado.
Conjunta	$P(A \text{ e } B) = \text{count}(A \text{ e } B) / N$	Suporte da regra.
Condicional	$P(A B) = P(A \text{ e } B) / P(B)$	Pergunta feita pelo usuario.
Confianca	$\text{confidence}(B \rightarrow A) = P(A B)$	Precisao da regra.
Lift	$\text{lift} = P(A B) / P(A)$	Forca da associacao.
Intervalo	$\text{round}(p,3) \pm 0,001$	Probabilidade intervalar para evitar rigidez numerica.
Variavel LP	$x_w \geq 0$	Probabilidade de cada mundo possivel.
Normalizacao	$\text{soma}(x_w) = 1$	Distribuicao de probabilidade valida.
Restricao	$L \leq \text{soma}(x_w \text{ onde } E) \leq U$	Conhecimento probabilistico da base.
Charnes-Cooper	$P(A \text{ e } B)/P(B) \rightarrow \text{objetivo linear}$	Permite resolver condicional com linprog.

6.1 Fundamentacao explicada

A fundamentacao do codigo parte da ideia de que uma base categorica pode ser vista como fonte empirica de conhecimento probabilistico. Cada frequencia observada no dataset fornece uma estimativa de probabilidade. Em vez de usar essas estimativas apenas como valores isolados, o projeto as transforma em restricoes lineares. Assim, o solver nao calcula uma unica resposta direta; ele calcula os menores e maiores valores de $P(A | B)$ que ainda sao compatíveis com as evidencias extraídas da base.

A variavel x_w representa a probabilidade de um mundo possivel w . Um mundo possivel e uma combinacao categorica observada ou representada a partir da base. A restricao $\text{soma}(x_w)=1$ garante que todas essas massas formem uma distribuicao de probabilidade. As restricoes de marginais e conjuntas, como $P(A)$, $P(B)$ e $P(A \text{ e } B)$, limitam a soma das variaveis x_w que satisfazem cada evento.

As regras de associacao sao interpretadas como $B \rightarrow A$. O suporte mede $P(A \text{ e } B)$, a confianca mede $P(A | B)$ e o lift compara $P(A | B)$ com $P(A)$. Quando o suporte conjunto e zero, a consulta pode ser exibida ao usuario, mas nao deve ser inserida como restricao ativa de confianca, pois o dataset nao aprendeu evidencia positiva para aquela regra.

Como $P(A | B)=P(A \text{ e } B)/P(B)$ e uma razao, a consulta nao entra diretamente como objetivo linear. Por isso foi aplicada a transformacao de Charnes-Cooper: a razao e reescrita com variaveis transformadas, fixa-se a massa de B e o solver consegue minimizar e maximizar a probabilidade condicional em formato linear.

7. Solver

O solver nao e separado do projeto. Ele e uma dependencia instalada pelo requirements.txt. O backend importa `scipy.optimize.linprog` e executa o solver dentro da propria aplicacao. No Render, o deploy instala SciPy e Gunicorn; depois inicia o Flask app.

```
scipy.optimize.linprog(method='highs')
```

8. Saidas do sistema

Saida	Conteudo
Interface	Metricas, intervalo linear, programa linear e conclusao automatica.

Saida	Conteudo
API /api/query	JSON dinamico com probabilidades, solver, tempo e conclusao.
relatorio_saida_programacao_linear.pdf	Graficos, calculos, algoritmos e explicacao.
saida_calculada_programacao_linear.json	Calculos numericos dos algoritmos.
roadmap_base_cientifica.pdf	Roadmap e base cientifica/técnica.
conclusao_resultados_programacao_linear.pdf	Conclusao interpretativa dos resultados.
contexto_completo_projeto.pdf	Este documento com o contexto total do projeto.

9. Tratamento de casos especiais

Quando $P(B)=0$, a consulta condicional não é matematicamente definida, pois haveria divisão por zero. O sistema detecta esse caso antes do solver, explica a inviabilidade e orienta o usuário a reduzir as condições ou escolher uma combinação existente no dataset.

10. Deploy no Render

O projeto foi preparado como Web Service Python no Render. A versão Python foi fixada em 3.11.9 para evitar falha de instalação do SciPy no Python 3.14. O start correto é `gunicorn app:app`, mas também foi criado um módulo `your_application.wsgi` para compatibilidade com start command antigo.

```
Build: python -m pip install --upgrade pip setuptools wheel && python -m pip install -r
requirements.txt
Start: gunicorn app:app --bind 0.0.0.0:$PORT --timeout 120
Health check: /healthz
```

11. Base científica

O projeto se apoia em raciocínio sob incerteza, probabilidades intervalares, regras de associação, estimativas por frequência/verossimilhança e programação linear. Os links indicados pelo professor foram usados para justificar a formulação por restrições, o uso de intervalos, a possibilidade de smoothing e a comparação futura de solvers.

12. Limites e próximos passos

O sistema já responde consultas condicionais e gera conclusões. Como próximos passos, é possível adicionar Laplace smoothing na interface, gerar conjuntas de pares/trios como nível opcional de restrição, comparar vários solvers, medir curvas de tempo e calcular acurácia quando o atributo perguntado for uma classe.

13. Conclusão final

Conclui-se que o projeto implementa o ciclo completo solicitado: extração de conhecimento probabilístico, transformação em restrições lineares, pergunta condicional do usuário, resolução com solver e apresentação interpretável do resultado. A aplicação também registra tempos de processamento, gera relatórios em PDF/JSON e está pronta para deploy no Render.